计算机系统基础实验报告

Lab2：BombLab

课程： 计算机系统基础（2026 秋）

实验： Lab2 BombLab

姓名： 赵静怡

学号： 25800680034

环境： Windows 11 / WSL Ubuntu / x86-64

代码仓库：https://github.com/zhaojy200728/ics-2026-bomblab

目录

1 实验内容与环境 2

2 Phase 1：函数调用与返回值 2

3 Phase 2：数组与循环递推 4

4 Phase 3：跳转表与分支计算 5

5 Phase 4：递归搜索与返回值计算 7

6 Phase 5：链表结构与遍历 9

7 Phase 6：函数指针与间接调用 12

8 Honor：输入布局与校验算法 14

9 测试结果与提交检查 16

10 实验总结 18

11 参考资料与工具使用说明 18

致谢 18

1 实验内容与环境

本实验给出个人编译版本的 ELF 可执行程序 `bomb`。任务是分析六个正式关卡及 Honor 的输入验证过程，在 `defuser.c` 中实现根据 challenge 计算口令的通用求解函数，并通过公开测试和不同 seed 的实际运行验证。个人炸弹不包含正式关卡的 C 源代码，主要分析对象是二进制反汇编及运行时内存数据。

实验在 Windows 11 的 WSL Ubuntu x86-64 环境进行，使用 VS Code 编辑，GCC 编译 C11 程序，GNU `objdump` 查看反汇编，GDB 设置断点并读取寄存器、栈帧和静态数据区。

1.1 Tutor 调试练习

正式分析之前，在 `tutor/` 目录编译并运行 Phase 0，`make test` 返回 10/10。反汇编 `phase_00` 后，使用 GDB 在函数入口设置断点，执行 `x/s $rdi` 查看口令字符串，使用 `p/x $rsi` 查看 challenge。计算关系为：

$$a_0 = ((challenge \gg 5)\ \&\ 255) \oplus 42, \qquad a_1 = 3a_0 + 7.$$

通过 tutor 练习，我对断点、参数寄存器以及 seed 与 challenge 的关系等有了初步理解。

1.2 逆向分析方法

各关采用相同的基本检查流程：根据 `valid_u32_fields` 确定输入整数个数；结合 `sscanf` 与栈偏移定位输入变量；追踪 `cmp` 的比较对象确定期望值来源；反汇编被调用的辅助函数；对于跳转表、递归或静态数据，使用 GDB 进一步确认；将验证计算转写为 C 语言，并在本地编译测试。

反汇编采用 AT&T 语法，操作数通常按源、目的顺序排列。由于程序多使用 32 位寄存器参与计算，C 实现中采用 `uint32_t` 保持无符号回绕语义。静态分析用于确认指令与控制流程，动态调试用于检查指定指令处的实际数值和内存。

2 Phase 1：函数调用与返回值

2.1 输入格式与目标值定位

`phase_01` 调用 `valid_u32_fields` 时传入参数 1，`sscanf` 的返回值也与 1 比较，因此输入格式为一个无符号整数。函数完成输入解析后执行以下指令：

```
mov %r12,%rdi
call 402050 <p1_compute>
cmp  %eax,-0x14(%rbp)
sete %al
```

`p1_compute` 返回第一关期望的整数。继续反汇编可见，它对 challenge 分别取低 32 位、右移 16 位和右移 32 位，再将对应数据传入 `p1_step0/1/2`。这些函数使用 `lea`、`xor`、`add` 计算，其中 `lea (%rdi,%rsi,4),%eax` 等价于 $x + 4y$。

令

$$L = (uint32_t)challenge,$$
$$M = (uint32_t)(challenge \gg 16),$$
$$H = (uint32_t)(challenge \gg 32).$$

三个辅助函数对应：

$$A = (H + 4M) \oplus \texttt{0x2BCD},$$
$$B = (H + \texttt{0x8D05}) \oplus L,$$
$$answer = (3B + A) \oplus \texttt{0x6332}.$$

`p1_compute` 依次调用 `p1_step1`、`p1_step0` 和 `p1_step2`，中间结果通过寄存器传递。最终表达式按汇编中的实际调用顺序整理。

2.2 算法实现与测试

```
uint32_t lo = (uint32_t)challenge;
uint32_t mid = (uint32_t)(challenge >> 16);
uint32_t hi = (uint32_t)(challenge >> 32);
uint32_t a = (hi + 4u * mid) ^ 0x2BCDu;
uint32_t b = (hi + 0x8D05u) ^ lo;
values[0] = (3u * b + a) ^ 0x6332u;
count = 1;
```

代码写入 `solve_phase_1`，编译后十组公开 seed 全部通过。使用 seed 42 分别运行 `defuser` 和 `bomb`，将求解器输出输入炸弹，显示 `Phase 1 defused.`。

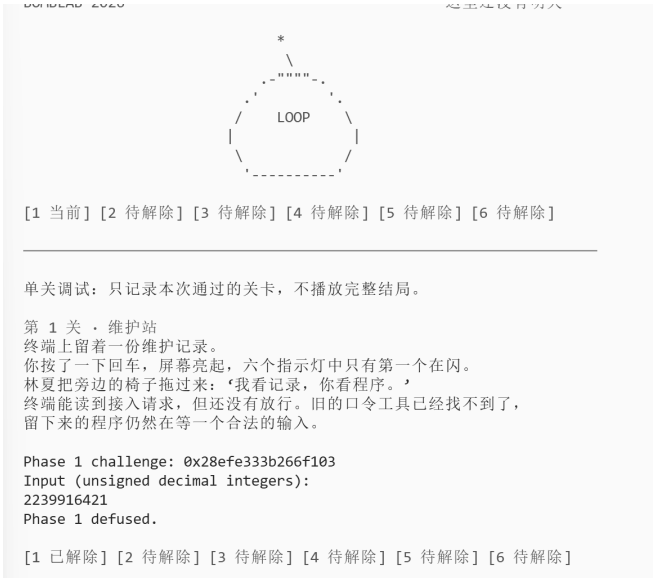


图 1: 第一关成功解除。

3 Phase 2：数组与循环递推

3.1 数组访问和循环结构

第二关的 `valid_u32_fields` 参数为 6，`sscanf` 将六个整数写入栈空间。汇编采用栈相对寻址读取数组元素，其中 `%rdx` 为循环下标，比例系数 4 对应 32 位整数的字节数。

```
mov $0x9,%ecx
mov $0x1,%edx
...                          # repeat at 0x402178
shr %cl,%rax
and $0x1f,%eax
add %edx,%eax
xor %esi,%eax
mov -0x40(%rbp,%rdx,4),%esi
lea (%rax,%rax,2),%eax
cmp %eax,%esi
add $0x1,%rdx
add $0x2,%ecx
cmp $0x6,%rdx
jne 402178
```

循环下标 `%rdx` 从 1 增至 5，`%ecx` 从 9 开始每轮加 2，移位次数依次为 9、11、13、15、17。根据前一项与 challenge 位域的异或结果，使用 `lea` 乘以 3 得到下一项，因此递推关系为：

$$
\begin{aligned}
a_0 &= ((challenge \gg 13)\ \&\ 127) \oplus 28, \\
a_i &= 3\Big(a_{i-1} \oplus \big[((challenge \gg (2i+7))\ \&\ 31) + i\big]\Big), \quad i = 1, \ldots, 5.
\end{aligned}
$$

```
values[0] = (uint32_t)((challenge >> 13) & 0x7fULL) ^ 0x1cu;
for (uint32_t i = 1; i < 6; i++) {
    uint32_t shift = 2u * i + 7u;
    uint32_t t = (uint32_t)((challenge >> shift) & 0x1fULL);
    values[i] = 3u * ((t + i) ^ values[i - 1]);
}
count = 6;
```

3.2 GDB 断点验证

在 `0x40216a` 处设置断点，用六个 0 作为占位输入。GDB 显示 `p/u $eax` 为 96，`p/u $esi` 为 0。对应 seed 42 时，求解器输出第一项也为 96，确认了第一个元素的计算结果。

```
Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):

Breakpoint 1, 0x000000000040216a in phase_02 ()
(gdb) p/u $eax
$1 = 96
(gdb) p/u $esi
$2 = 0
(gdb)
```

图 2: Phase 2 的比较指令处读取到 eax = 96、esi = 0。

六个整数在 seed 42 下通过交互验证，公开测试结果为 10/10。

```
96 336 975 2928 8697 26079
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$ python3 public_test.py --phase 2
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$
```

图 3: Phase 2 的公开测试结果

```
单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
96 336 975 2928 8697 26079
Phase 2 defused.

[1 待解除] [2 已解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

图 4: Phase 2 解除结果

4 Phase 3：跳转表与分支计算

4.1 分支参数与跳转表

`phase_03` 要求两个整数。第一项与 `challenge & 7` 比较，所以是 $0 \sim 7$ 的分支编号。第二项来自 `p3_dispatch`，其参数是

$$index = (uint32_t)challenge \,\&\, 7,$$
$$x = (uint32_t)(challenge \gg 3) \oplus (uint32_t)(challenge \gg 32).$$

在 `p3_dispatch` 里出现了第三关的关键指令：

```
cmp $0x7,%eax
ja  4022c5
notrack jmp *0x405820(,%rax,8)
```

它依据 %rax 的值从 0x405820+8*index 读取一个 64 位代码地址并跳转，正是 switch 跳转表的底层实现。

p3_case_0 至 p3_case_7 的函数编号与跳转表下标并不完全一致。使用 GDB 执行 x/8gx 0x405820，读取到的地址为：

$$(\texttt{402298}, \texttt{4022a0}, \texttt{4022a8}, \texttt{4022b0}, \texttt{4022b8}, \texttt{4022c0}, \texttt{402288}, \texttt{402290}).$$

将地址与函数符号对应，得到索引映射 $[2, 5, 7, 1, 6, 3, 4, 0]$。C 语言实现按这一映射选择分支。

```
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bomb...
(No debugging symbols found in ./bomb)
(gdb) starti
Starting program: /home/zhaojy/ics-2026-bomblab/bomb

Program stopped.
0x00007ffff7fe3340 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) x/8gx 0x405820
0x405820:       0x0000000000402298      0x00000000004022a0
0x405830:       0x00000000004022a8      0x00000000004022b0
0x405840:       0x00000000004022b8      0x00000000004022c0
0x405850:       0x0000000000402288      0x0000000000402290
(gdb) quit
```

图 5: GDB 读取到的八项跳转地址

4.2 分支算法与实现

把 p3_case_0 至 p3_case_7 的指令逐个翻译，可得到下表（均按 32 位无符号算术计算）。

输入 $index$	执行函数	第二个数 $answer_1$
0	case_2	$(x \oplus \texttt{0x5AC9}) + (x \gg 2)$
1	case_5	$5x + \texttt{0x6A44}$
2	case_7	$(x \oplus \texttt{0x304A}) + (x \gg 2)$
3	case_1	$5x + \texttt{0x49C14}$
4	case_6	$(4x) \oplus \texttt{0x7B30}$
5	case_3	$5x + \texttt{0x12EC}$
6	case_4	$5x + \texttt{0x28C26}$
7	case_0	$(4x) \oplus \texttt{0x1E6E}$

在 C 中使用 switch(index) 按上述映射输出第二个整数：

```
values[0] = (uint32_t)challenge & 7u;
uint32_t x = (uint32_t)(challenge >> 3)
           ^ (uint32_t)(challenge >> 32);
switch (values[0]) {
case 0: values[1] = (x ^ 0x5ac9u) + (x >> 2); break;
case 1: values[1] = 5u * x + 0x6a44u; break;
case 2: values[1] = (x ^ 0x304au) + (x >> 2); break;
case 3: values[1] = 5u * x + 0x49c14u; break;
case 4: values[1] = (4u * x) ^ 0x7b30u; break;
```

```
case 5: values[1] = 5u * x + 0x12ecu; break;
case 6: values[1] = 5u * x + 0x28c26u; break;
case 7: values[1] = (4u * x) ^ 0x1e6eu; break;
}
count = 2;
```

第三关公开测试 10/10，seed 42 单关运行显示 `Phase 3 defused.`。

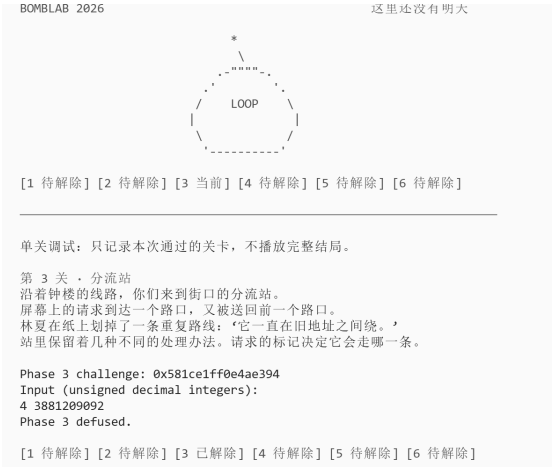


图 6: Phase 3 通关结果

5 Phase 4：递归搜索与返回值计算

5.1 参数与递归边界

`phase_04` 输入一个整数，调用 `p4_walk` 前分别准备四个参数：

$$\begin{aligned} target &= challenge \,\&\, 127, \\ low &= 0, \quad high = 127, \\ extra &= (uint32_t)((challenge \gg 7) \oplus (challenge \gg 32)). \end{aligned}$$

`p4_walk` 根据 `high-low`、`shr 1`、`add low` 计算中点：

$$mid = low + \left\lfloor \frac{high - low}{2} \right\rfloor .$$

`cmp` 后分为三条路径：等于中点时返回；目标小于中点时递归搜索左半区间；目标大于中点时递归搜索右半区间。

5.2 递归返回值

函数的返回值还包含递归路径上的运算。递归调用返回后执行：

```
lea (%rax,%rax,4),%eax    # 5 * child_result
xor %ebx,%eax            # xor (mid + extra)
...                       # branch-specific final xor
xor $0x8,%eax            # left branch
# another path: xor $0x1c,%eax   (right branch)
```

各分支在子调用结果上执行不同的异或操作，故返回值定义为：

$$W(t,l,h,e)=\begin{cases}(mid+\texttt{0xC2})\oplus e, & t=mid,\\ [5W(t,l,mid-1,e)\oplus(mid+e)]\oplus\texttt{0x8}, & t<mid,\\ [5W(t,mid+1,h,e)\oplus(mid+e)]\oplus\texttt{0x1C}, & t>mid.\end{cases}$$

```
static uint32_t walk(uint32_t t, uint32_t l,
                     uint32_t h, uint32_t extra) {
    uint32_t mid = l + ((h - l) >> 1);
    if (t == mid) return (mid + 0xc2u) ^ extra;
    uint32_t mix = mid + extra;
    if (t < mid)
        return (5u * walk(t, l, mid - 1, extra) ^ mix) ^ 0x8u;
    return (5u * walk(t, mid + 1, h, extra) ^ mix) ^ 0x1cu;
}
values[0] = walk(target, 0, 127, extra);
count = 1;
```

5.3 调用栈观察与测试

在 `p4_walk` 函数入口设置断点。seed 42 的首层参数为 $target=114$、$low=0$、$high=127$。继续执行到下一层时，`backtrace` 显示两层 `p4_walk`，其下为 `phase_04` 和 `main`，确认存在嵌套递归调用。

```
Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):

Breakpoint 1, 0x0000000000402350 in p4_walk ()
(gdb) info registers edi esi edx ecx
edi            0x72                114
esi            0x0                 0
edx            0x7f                127
ecx            0xbdf4c81d          -1108031459
(gdb) continue
Continuing.

Breakpoint 1, 0x0000000000402350 in p4_walk ()
(gdb) backtrace
#0  0x0000000000402350 in p4_walk ()
#1  0x000000000040237c in p4_walk ()
#2  0x000000000040242a in phase_04 ()
#3  0x00000000004013fc in main ()
(gdb)
```

图 7: Phase 4 的实测寄存器和 backtrace，递归函数在调用栈上的层次。

公开测试 10/10，seed 42 单关输入通过。

```
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
2898096151
Phase 4 defused.

[1 待解除] [2 待解除] [3 待解除] [4 已解除] [5 待解除] [6 待解除]
```

图 8: Phase 4 解除结果。

6 Phase 5：链表结构与遍历

6.1 节点结构与指针偏移

`phase_05` 检查两个整数。真正的计算在 `p5_traverse` 中：它从 `0x405b60` 的八项入口表取出指针，读取偏移 8、12 处的两个字段，再通过 `mov (%rdx),%rdx` 跳到下一个节点。

```
mov 0x405b60(,%rax,8),%rdx  # select head pointer
mov 0x8(%rdx),%esi          # value
add 0xc(%rdx),%eax          # salt
mov (%rdx),%rdx             # follow next pointer
```

由此恢复出每个节点 16 字节的布局：

```
typedef struct Node {
    struct Node *next;   // +0, 8 bytes
    uint32_t value;      // +8, 4 bytes
    uint32_t salt;       // +12, 4 bytes
} Node;
```

使用 GDB 的 `x/8gx 0x405b60` 读取八个入口，通过 `x/1gx $node` 和 `x/2uw $node+8` 检查单个节点。举例来说，地址 `0x405bd0` 的 `next=0x405c50`、`value=2158`、`salt=18939`；下一个节点中的 `value=1598`、`salt=16897`。通过上述地址关系确认了节点的指针字段与数值字段。

```
(gdb) set pagination off
(gdb) starti
Starting program: /home/zhaojy/ics-2026-bomblab/bomb

Program stopped.
0x00007ffff7fe3340 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) x/8gx 0x405b60
0x405b60 <p5_heads>:    0x0000000000405bd0      0x0000000000405c50
0x405b70 <p5_heads+16>: 0x0000000000405bf0      0x0000000000405c30
0x405b80 <p5_heads+32>: 0x0000000000405c00      0x0000000000405bc0
0x405b90 <p5_heads+48>: 0x0000000000405bb0      0x0000000000405c10
(gdb) set $node = *(unsigned long long *)0x405b60
(gdb) x/1gx $node
0x405bd0 <p5_nodes+48>: 0x0000000000405c50
(gdb) x/2uw $node+8
0x405bd8 <p5_nodes+56>: 2158    18939
(gdb) set $next = *(unsigned long long *)$node
(gdb) x/1gx $next
0x405c50 <p5_nodes+176>:        0x0000000000405bf0
(gdb) x/2uw $next+8
0x405c58 <p5_nodes+184>:        1598    16897
(gdb)
```

图 9: 通过 GDB 核对头节点指针以及相邻节点的 value、salt 字段。

6.2 节点数据恢复

`x/48wx 0x405ba0` 读取了 12 个节点共 192 字节，按 16 字节分组，每组前 8 字节是 next，后 8 字节依次是 value 与 salt。将绝对地址换成数组下标后，得到以下完整数据：

节点下标	next 下标	value（十六进制）	salt（十六进制）
0	4	0164	FAE7
1	7	0304	2FB0
2	1	0A1C	F060
3	11	086E	49FB
4	8	0438	33C4
5	9	052B	881E
6	2	0730	3480
7	10	0B6F	FA97
8	终点	0934	A8F9
9	6	0046	8A31
10	0	020C	86B3
11	5	063E	4201

八个头节点对应的下标是 $[3, 11, 5, 9, 6, 2, 1, 7]$。遍历顺序由 `next` 字段决定，与节点在静态数组中的排列顺序不同。

```
(gdb) x/48wx 0x405ba0
0x405ba0 <p5_nodes>:    0x00405be0      0x00000000      0x00000164      0x0000fa
e7
0x405bb0 <p5_nodes+16>: 0x00405c10      0x00000000      0x00000304      0x00002f
b0
0x405bc0 <p5_nodes+32>: 0x00405bb0      0x00000000      0x00000a1c      0x0000f0
60
0x405bd0 <p5_nodes+48>: 0x00405c50      0x00000000      0x0000086e      0x000049
fb
0x405be0 <p5_nodes+64>: 0x00405c20      0x00000000      0x00000438      0x000033
c4
0x405bf0 <p5_nodes+80>: 0x00405c30      0x00000000      0x0000052b      0x000088
1e
0x405c00 <p5_nodes+96>: 0x00405bc0      0x00000000      0x00000730      0x000034
80
0x405c10 <p5_nodes+112>:        0x00405c40      0x00000000      0x00000b6f      0
x0000fa97
0x405c20 <p5_nodes+128>:        0x00000000      0x00000000      0x00000934      0
x0000a8f9
0x405c30 <p5_nodes+144>:        0x00405c00      0x00000000      0x00000046      0
x00008a31
0x405c40 <p5_nodes+160>:        0x00405ba0      0x00000000      0x0000020c      0
x000086b3
0x405c50 <p5_nodes+176>:        0x00405bf0      0x00000000      0x0000063e      0
x00004201
(gdb)
```

图 10: 12 个节点的原始内存转储；图中的地址与上表逐项对应。

6.3 遍历算法与实现

由 `p5_traverse` 中的移位和掩码得到：

$$index = challenge \,\&\, 7, \qquad steps = (challenge \gg 3) \,\&\, 3,$$

$$acc_0 = ((uint32_t)(challenge \gg 6) \oplus (uint32_t)(challenge \gg 32)) \oplus \texttt{0xACE4}.$$

从 `heads[index]` 开始，共访问 $steps + 1$ 个节点。每次以当前节点的两个字段更新：

$$acc_{k+1} = (3acc_k + salt_k) \oplus value_k.$$

`phase_05` 的第一个输入是最终 `acc`，第二个输入是*最后访问节点的value*，这还可以从调用 `p5_traverse` 时传入局部变量地址、返回后比较两个栈上值来验证。独立求解程序不能直接读取另一个进程的链表地址，因此将节点值和连接关系恢复成 `static const` 数据表。

```
typedef struct {
    int next;
    uint32_t value, salt;
} NodeData;
static const NodeData nodes[12] = {
    {4,0x164u,0xfae7u}, {7,0x304u,0x2fb0u},
    {1,0xa1cu,0xf060u}, {11,0x86eu,0x49fbu},
    {8,0x438u,0x33c4u}, {9,0x52bu,0x881eu},
    {2,0x730u,0x3480u}, {10,0xb6fu,0xfa97u},
    {-1,0x934u,0xa8f9u}, {6,0x46u,0x8a31u},
    {0,0x20cu,0x86b3u}, {5,0x63eu,0x4201u}
};
static const int heads[8] = {3,11,5,9,6,2,1,7};
uint32_t idx = (uint32_t)(challenge & 7u);
uint32_t steps = (uint32_t)((challenge >> 3) & 3u);
uint32_t acc = ((uint32_t)(challenge >> 6)
             ^ (uint32_t)(challenge >> 32)) ^ 0xace4u;
int node_id = heads[idx];
```

```
for (uint32_t k = 0; k <= steps; ++k) {
    const NodeData *node = &nodes[node_id];
    acc = (3u * acc + node->salt) ^ node->value;
    values[1] = node->value;
    if (k < steps) node_id = node->next;
}
values[0] = acc;
count = 2;
```

其中 NodeData 用整数下标替代原炸弹内存中的绝对指针，12 组常量都已由 GDB 转储还原；实现只依赖本地静态表，不使用运行中炸弹的地址。公开测试 10/10，seed 42 手动通关成功。



图 11: Phase 5 通关结果

7 Phase 6：函数指针与间接调用

7.1 函数指针表与调用顺序

第六关只需一个整数。phase_06 把 challenge 传给 p6_dispatch；在后者中看见了两条 call *...:

```
and $0x38,%ebx
and $0x7,%eax
call *0x405e00(,%rax,8)
xor  $0x2269,%eax
mov  %eax,%edi
call *0x405e00(%rbx)
add  $0xeda3,%eax
```

令

$$x = (uint32_t)((challenge \gg 6) \oplus (challenge \gg 32)),$$
$$i = challenge \,\&\, 7, \quad j = (challenge \gg 3) \,\&\, 7.$$

程序先调用 ops[i](x)，把结果异或 0x2269 后，再调用 ops[j]，最后加 0xEDA3。这里 %rbx 的值经过 & 0x38，已经是索引乘以 8 的字节偏移，所以第二条 call 没再写显式的缩放系数。

GDB 的 x/8gx 0x405e00 读出实际函数地址，再用 nm -n ./bomb | grep p6_ 与符号对应，得到八项函数映射：$[0, 0, 4, 3, 1, 3, 5, 2]$。指针表中存在重复项，八个表项实际对应六个运算函数。

```
(gdb) starti
Starting program: /home/zhaojy/ics-2026-bomblab/bomb

Program stopped.
0x00007ffff7fe3340 in _start () from /lib64/ld-linux-x86-64.so.2
(gdb) x/8gx 0x405e00
0x405e00 <p6_ops>:      0x0000000000402520      0x0000000000402520
0x405e10 <p6_ops+16>:   0x0000000000402590      0x0000000000402570
0x405e20 <p6_ops+32>:   0x0000000000402540      0x0000000000402570
0x405e30 <p6_ops+48>:   0x00000000004025a0      0x0000000000402560
(gdb) quit
A debugging session is active.

        Inferior 1 [process 66442] will be killed.

Quit anyway? (y or n) y
zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$ nm -n ./bomb | grep 'p6_'
0000000000402520 T p6_op_0
0000000000402540 T p6_op_1
0000000000402560 T p6_op_2
0000000000402570 T p6_op_3
0000000000402590 T p6_op_4
00000000004025a0 T p6_op_5
00000000004025b0 T p6_dispatch
0000000000405e00 r p6_ops
```

图 12: 函数指针表与符号地址的对照

7.2 运算函数与实现

从 `0x402520` 到 `0x4025b0` 反汇编六个短函数，恢复为：

函数	对输入 x 的运算
op_0	$(x \oplus \texttt{0x4674}) + (x \ll 7)$
op_1	$(8x + \texttt{0xA8CA}) \oplus (7x)$
op_2	$(x \gg 7) \mid (x \ll 25)$，循环右移 7 位
op_3	$((x \gg 7) \oplus x) + \texttt{0xAC7B}$
op_4	$(x + \texttt{0xF547}) \oplus (x \gg 5)$
op_5	$((x \ll 1) \mid (x \gg 31)) \oplus \texttt{0x3C41}$，循环左移 1 位

`ror` 和 `rol` 分别执行循环右移与循环左移。C 实现通过两个无符号移位及按位或组合，保持 32 位运算语义。

```
static uint32_t apply(unsigned op, uint32_t x) {
    switch (op) {
    case 0: return (x ^ 0x4674u) + (x << 7);
    case 1: return (8u*x + 0xa8cau) ^ (7u*x);
    case 2: return (x >> 7) | (x << 25);  // ror 7
    case 3: return ((x >> 7) ^ x) + 0xac7bu;
    case 4: return (x + 0xf547u) ^ (x >> 5);
    case 5: return ((x << 1) | (x >> 31)) ^ 0x3c41u; // rol 1
    default: return 0;
    }
}
static const unsigned ops[8] = {0,0,4,3,1,3,5,2};
uint32_t result = apply(ops[i], x) ^ 0x2269u;
values[0] = apply(ops[j], result) + 0xeda3u;
count = 1;
```

第六关公开测试为 10/10，seed 42 手动运行显示 `Phase 6 defused.`。六个正式关卡均已在 `defuser.c` 中实现通用解法。



图 13: Phase 6 通关结果

8 Honor：输入布局与校验算法

8.1 输入长度与内存偏移

Honor 的输入为第七行 ASCII 文本。运行时使用同一个 seed 完成前六关，再通过管道将第七行传入炸弹。相关函数包括 `secret_gate`（入口检查）、`secret_copy`（复制字符串）和 `secret_phase`（数值校验）。

在 `secret_gate` 中，`strlen` 返回值减去 `0x24` 后与 `0x7f` 无符号比较，要求总长度在 36 到 163 字节之间；`secret_copy` 从源地址逐字节复制到目标缓冲区，直到复制包含结尾零字节的整段字符串。

需要关注以下三个栈帧位置：

```
-0xd0(%rbp)    # secret_copy destination
-0xb0(%rbp)    # compared with 0x4E45504F
-0xac(%rbp)    # pointer passed to secret_phase
```

后两处距离复制起点分别为 32 和 36 字节。按小端序，检查常量对应字符 `OPEN`。

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPENa b c
```

输入前 32 字节为填充内容，偏移 32–35 为 `OPEN`，偏移 36 之后为三个以空格分隔的无符号整数。

8.2 数值校验公式

`secret_phase` 先检查第三段文本中恰好有 3 个合法的无符号整数。令 $C = challenge$、$T = context$，定义

$$X = (uint32_t)C \oplus (uint32_t)(C \gg 32), \qquad Y = (uint32_t)T \oplus (uint32_t)(T \gg 32).$$

`secret_mix` 的指令

```
lea 0x393f(%rdi,%rdi,2),%eax
xor $0x8435,%eax
ret
```

直接翻译为 $mix(z) = (3z + \texttt{0x393F}) \oplus \texttt{0x8435}$。再沿着三次 `cmp` 追踪，三个整数分别为：

$$
\begin{aligned}
a &= (uint32_t)T \,\&\, 3, \\
b &= mix(X \oplus Y) \oplus \Big((\texttt{0x393F} \cdot a) \oplus (uint32_t)(T \gg 16)\Big), \\
c &= b \oplus \Big(Y + a + \texttt{0x8435} + (X \gg 7)\Big).
\end{aligned}
$$

其中 `context` 由前六行口令经公共驱动程序生成，所以 `solve_secret` 直接使用其入参，不能把某次运行的 context 写死。

```
uint32_t X = (uint32_t)challenge ^ (uint32_t)(challenge >> 32);
uint32_t Y = (uint32_t)context   ^ (uint32_t)(context >> 32);
uint32_t a = (uint32_t)context & 3u;
uint32_t mixed = (3u * (X ^ Y) + 0x393fu) ^ 0x8435u;
uint32_t b = mixed ^ ((0x393fu * a)
                      ^ (uint32_t)(context >> 16));
uint32_t c = b ^ (Y + a + 0x8435u + (X >> 7));
const char *prefix = "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN";
int n = snprintf(out, cap, "%s%" PRIu32 " %" PRIu32 " %" PRIu32,
                 prefix, a, b, c);
return n < 0 || (size_t)n >= cap;
```

8.3 运行与测试

`python3 public_test.py --secret` 的十组公开 seed 全部显示 PASS。完整运行时，用同一个 seed 先输出六行、再输出 Honor 的第七行，通过管道送进炸弹：

```
{ ./defuser --seed 42; ./defuser --secret --seed 42; } | ./bomb --seed 42
```

完整运行后显示 `Secret phase defused.` 和 `[SAFE]`。

```
zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$ make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solu
tion.c challenge.c cli.c -o defuser
zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
zhaojy@LAPTOP-OC5V7TC3:~/ics-2026-bomblab$ { ./defuser --seed 42; ./defuser --secret --seed 42;
} | ./bomb --seed 42
```

图 14: Honor 的 10 组公开 seed 测试全部通过。

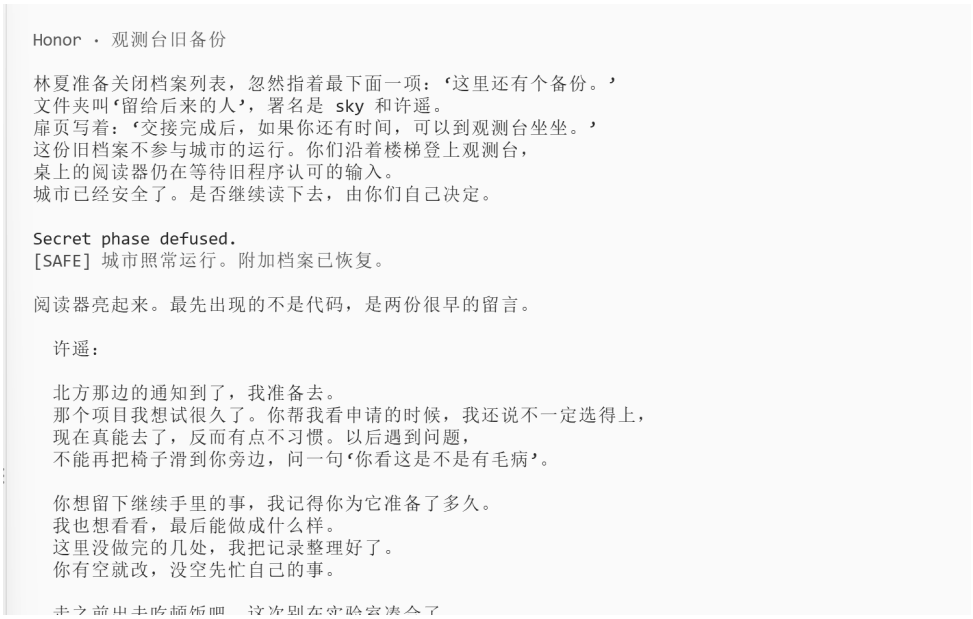


图 15: 完整运行中 Honor 的通过结果。

9 测试结果与提交检查

9.1 公开测试与单关运行

各关实现后使用 `make` 编译，运行对应公开测试，并通过 `--seed 42 --phase N` 手动输入计算出的口令。六个正式关卡的公开测试均为 10/10，Honor 公开测试也为 10/10。

关卡	公开测试	seed 42 单关交互	关键观察
Phase 1	10/10	通过	辅助函数返回值
Phase 2	10/10	通过	GDB 比较寄存器
Phase 3	10/10	通过	跳转表内存
Phase 4	10/10	通过	递归 backtrace
Phase 5	10/10	通过	链表节点内存
Phase 6	10/10	通过	函数指针表
Honor	10/10	整体通过	输入内存布局

公开测试共通过 $6 \times 10 + 10 = 70$ 组。最新 GitHub Actions 的 `BombLab public tests` 工作流显示通过，分支为 `main`。最终评分仍以助教的隐藏测试为准。

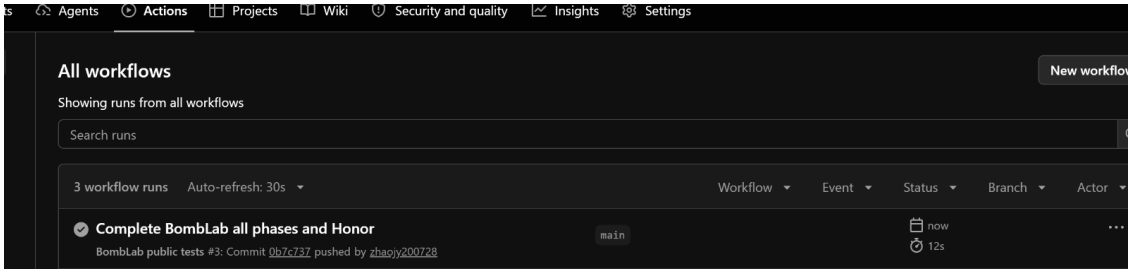


图 16: GitHub Actions 最新一次运行通过

9.2 额外 seed 验证

另外选取公开列表外的 seed 43，使用相同 seed 生成前六行及 Honor 口令，并进行完整运行。截图记录前六关及 Honor 均通过，测试命令如下：

```
{ ./defuser --seed 43; ./defuser --secret --seed 43; } | ./bomb --seed 43
```



图 17: 另一组 challenge 下的完整运行，前六关均显示 defused，随后进入 Honor。

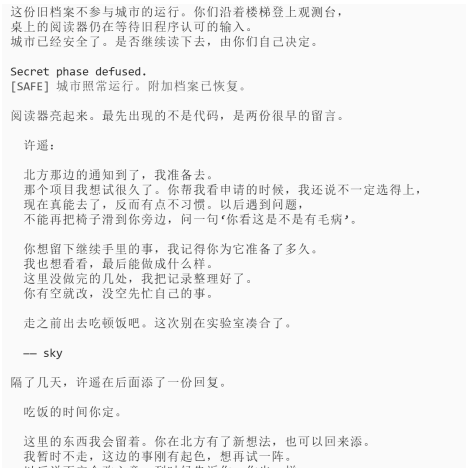


图 18: 额外 seed 自测中的 Honor 通过，后续剧情真的很赞!!!

该次测试提供了不同 challenge 下的运行记录，但公开测试与额外自测仍不能覆盖所有隐藏数据。

10 实验总结

本实验使用反汇编及 GDB 分析了六个正式关卡与 Honor 的输入验证逻辑，并在 `defuser.c` 中实现对应的 C 语言求解函数。各关涉及的主要知识点包括函数参数与返回值、数组递推、跳转表、递归调用、链表内存布局、函数指针表及字符串偏移校验。

在具体操作中，`objdump` 主要用于确定控制流程和计算表达式，GDB 用于确认跳转地址、调用栈和数据结构。例如，Phase 3 与 Phase 6 都需要读取静态地址表；Phase 4 通过 `backtrace` 检查

递归层数；Phase 5 将实际内存地址转换成独立求解器可使用的节点数据。对于数值运算，统一使用 32 位无符号整数实现移位、异或与溢出行为。

测试方面，六关及 Honor 的公开测试共通过 70 组，手动单关运行与完整运行均显示成功；额外使用 seed 43 完成一次完整验证。

11 参考资料与工具使用说明

[1] 复旦大学《计算机系统基础》2026 秋季，Lab2: BombLab 官方实验文档。

[2] 课程 BombLab 模板仓库 及 `tutor/` 示例。

[3] GNU GDB 文档，`https://www.sourceware.org/gdb/documentation/`。

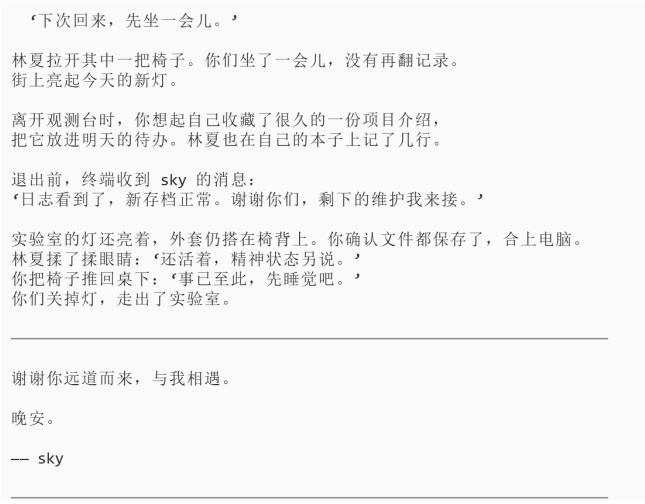
工具使用说明：使用 GNU objdump 反汇编个人炸弹，用 GDB 检查寄存器、调用栈及内存，用 GCC 和课程测试脚本编译验证解法。使用 ChatGPT 辅助理解部分汇编指令，并根据本地实际运行和测试记录进行核对。本文的运行截图来自实验过程，测试结果以本地终端和 GitHub Actions 的实际输出为依据。

致谢

想认真感谢设计和维护这次 BombLab 的老师、助教，以及写下《这里还没有明天》的创作者。六关的安排真的很用心，第一次在实验里感觉到温度，做到半夜，看到结局，我也想说：

谢谢你远道而来，与我相遇。

晚安。



‘下次回来，先坐一会儿。’

林夏拉开其中一把椅子。你们坐了一会儿，没有再翻记录。
街上亮起今天的新灯。

离开观测台时，你想起自己收藏了很久的一份项目介绍，
把它放进明天的待办。林夏也在自己的本子上记了几行。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

谢谢你远道而来，与我相遇。

晚安。

— sky

图 19: Honor 通关后的结尾文字。